

Las 4 Estrategias de Iteración — Plantillas de Prompt

Taller I · Día 2, Bloque 2 — y uso continuo el resto del taller Plan Institucional de Actualización
Docente · UABCS

Formato extraído del Documento 3.2 (Ficha Técnica: Las 4 Estrategias de Iteración).

Cómo se usa esta ficha: téngala a la mano mientras trabaja. Cuando su herramienta falle, identifique el síntoma en la Tabla de Decisión, vaya a la estrategia correspondiente y copie la plantilla sustituyendo los corchetes.

La regla general: describa el síntoma con precisión, no pida una solución genérica. *"Está mal, arréglalo"* obliga a la IA a adivinar — y adivina con la misma lógica que produjo el error.

Tabla de Decisión Rápida

Síntoma	Estrategia	Elemento clave del prompt
Mensaje de error en consola / notebook	1	El mensaje de error completo, sin parafrasear
Corre bien, pero el resultado matemático no es el esperado	2	Lo que aparece + lo que debería aparecer + caso de validación
Corre bien, es correcto, pero confunde al alumno	3	La mejora pedagógica concreta + "sin cambiar la lógica matemática"
Está terminado y quiero agregar algo nuevo	4	"Sin modificar lo existente" + descripción precisa de lo nuevo

Estrategia 1 — El código da error al ejecutar

Cuándo: el navegador o el notebook muestra un mensaje de error explícito. El programa ni siquiera llega a mostrar un resultado.

Plantilla:

```
Este código produce el siguiente error: [pegar el mensaje de error completo]

¿Qué lo causa y cómo se corrige?
Muestra el código corregido con comentarios explicando qué cambió y por qué.
```

Ejemplo (p5.js):

Este código produce el siguiente error:
"Uncaught TypeError: Cannot read properties of undefined (reading 'value')
at draw (sketch.js:23)"

¿Qué lo causa y cómo se corrige?
Muestra el código corregido con comentarios explicando qué cambió y por qué.

Nota práctica: en p5.js este error casi siempre significa que `draw()` intenta usar el slider antes de que `setup()` termine de crearlo, o que el nombre de la variable del slider no coincide entre ambas funciones. No hace falta saberlo para usar la estrategia, pero ayuda a reconocer si la corrección propuesta tiene sentido.

Estrategia 2 — El resultado corre pero es visualmente incorrecto

Cuándo: el programa ejecuta sin errores, pero lo que se ve no corresponde a lo que matemáticamente debería aparecer. Es el escenario típico de los tres errores comunes (escala, notación, casos límite).

Plantilla:

La gráfica muestra [descripción precisa de lo que aparece],
pero matemáticamente debería mostrar [descripción precisa de lo correcto].

Para verificar: si [parámetro] = [valor], el resultado exacto es [valor
calculado a mano].

El código actualmente muestra [lo que realmente aparece] para ese mismo caso.

¿Qué está mal?

Ejemplo (error de notación):

La gráfica muestra una función que oscila con periodo mucho más corto de lo esperado a medida que x crece, pareciendo acelerarse.

Pero matemáticamente debería mostrar $\sin^2(x)$, que es una función acotada entre 0 y 1, con periodo constante de π .

Para verificar: en $x = \pi/2$, $\sin^2(\pi/2) = 1$. El código actualmente muestra un valor distinto en ese punto.

¿Qué está mal?

Estrategia 3 — El resultado es correcto pero no es pedagógicamente útil

Cuándo: no hay error matemático. El programa hace bien lo que se pidió, pero algo no comunica: falta una etiqueta, el color no distingue dos elementos, el ritmo de la animación confunde, falta contexto visual.

Plantilla:

El programa funciona correctamente y el cálculo matemático es correcto.

Pero para mis alumnos sería más claro si [modificación pedagógica concreta].

Ajústalo sin cambiar la lógica matemática ni el cálculo existente.

Ejemplo:

El programa funciona correctamente: la matriz deforma la cuadrícula de manera correcta y los cuatro deslizadores actualizan la matriz en tiempo real.

Pero para mis alumnos sería más claro si el vector base $(1,0)$ y $(0,1)$ se mostraran en colores distintos y bien marcados antes y después de la transformación, para que puedan seguir visualmente hacia dónde se mueve cada uno.

Ajústalo sin cambiar la lógica matemática ni el cálculo existente.

⚠ **La última línea es un candado.** Sin ella, la IA suele reinterpretar la petición como permiso para tocar también el cálculo.

Estrategia 4 — Quiero añadir algo sin romper lo que ya funciona

Cuándo: la herramienta está terminada y validada, y se quiere extender con algo nuevo — un segundo método de comparación, un contador, otra forma de interacción — sin arriesgar lo probado.

Plantilla:

Sin modificar la funcionalidad existente, añade [nueva característica, descrita con precisión: qué hace, con qué variables interactúa, qué controla el usuario].

Ejemplo:

Sin modificar la funcionalidad existente (el campo de pendientes y la trayectoria trazada al hacer clic siguen funcionando igual), añade un segundo botón que permita comparar dos condiciones iniciales distintas al mismo tiempo, cada una en un color diferente, sin borrar la trayectoria anterior al hacer un nuevo clic.

⚠ **Riesgo específico:** es la estrategia más propensa a que la IA reescriba silenciosamente partes que ya funcionaban. Después de aplicarla, **repita el protocolo de validación completo** — no solo pruebe lo nuevo; vuelva a verificar el caso de validación original.

Práctica Aplicada

Elija una de las tres situaciones —idealmente distinta a la de su compañero más cercano—, identifique la estrategia y redacte el prompt.

Situación A. Un participante generó un simulador de lanzamiento de dados para probabilidad. Al ejecutar en el editor de p5.js, la consola muestra: `ReferenceError: numDados is not defined at rollDice (sketch.js:41)`.

Situación B. Un participante generó una herramienta de regresión lineal simple. El programa corre sin errores y el ajuste se ve razonable, pero al verificar a mano con tres puntos conocidos (1, 2), (2, 4), (3, 6) —una recta perfecta $y = 2x$ — la recta que traza el programa tiene pendiente visiblemente distinta de 2.

Situación C. Un participante tiene ya terminada y validada una herramienta sobre el método de Newton-Raphson. Funciona bien mostrando una iteración a la vez con un botón "siguiente". Ahora quiere que además se muestre un contador del número de iteraciones realizadas y el error absoluto respecto a la raíz conocida.

Situación elegida: ☐ A ☐ B ☐ C

Estrategia que aplica: ☐ 1 ☐ 2 ☐ 3 ☐ 4

Mi prompt de iteración:

Autorrevisión: ¿mi prompt incluye el elemento clave de esa estrategia (columna derecha de la Tabla de Decisión)? ☐ Sí ☐ Falta ajustarlo

Iterar no es fracasar. Ninguna de estas cuatro estrategias implica que el prompt original estuvo mal hecho. Incluso con una especificación excelente es común necesitar dos o tres rondas. Lo que distingue a un director eficiente no es acertar al primer intento, sino saber exactamente qué decirle a la IA cuando el primer intento no basta.

Formato para el participante · Taller I · UABCS